

MIP_KNOWLEDGE_GRAPH_MODEL.md

Mainframe Intelligence Platform (MIP)

Enterprise Knowledge Graph Architecture

Version: 1.0

Status: Core Architecture

Classification: Strategic Platform Asset

Owner: MIP Architecture Team

1. Executive Summary

The Knowledge Graph is the central intelligence layer of MIP.

Traditional modernization tools stop at:

- Source Code Discovery
- Program Inventory
- Dependency Reports

MIP continues further:

```
Repository
  ↓
Discovery
  ↓
Metadata
  ↓
Relationships
  ↓
Knowledge Graph
  ↓
Reasoning
  ↓
Impact Analysis
  ↓
Modernization Intelligence
  ↓
Transformation Roadmap
```

The Knowledge Graph enables engineers, architects, and leadership teams to understand large-scale legacy systems without manually reading thousands of programs.

2. Objectives

The Knowledge Graph must answer:

Discovery

- What programs exist?
- What jobs exist?
- What tables exist?
- What datasets exist?

Dependency Analysis

- Which program calls this program?
- Which jobs execute this program?
- Which datasets are consumed?
- Which tables are updated?

Impact Analysis

- What breaks if this program changes?
- What downstream systems are affected?
- Which batch jobs are impacted?

Modernization Intelligence

- Which components belong together?
 - What business capability does this support?
 - What service boundaries exist?
 - What should be modernized first?
-

3. Architecture Principles

Graph First

Everything becomes:

- Node
 - Relationship
-

Evidence Based

Every relationship must have evidence.

Example:

TS2AUTH CALLS TS2VALID

Evidence:
AUTHDRV.cbl
Line 120

Explainable

Every recommendation must be reproducible.

Incrementally Enriched

Graph maturity grows over time.

Phase 1

Programs
Jobs
Copybooks

Phase 2

DB2
VSAM
Datasets

Phase 3

Capabilities
Domains
Services

Phase 4

Modernization Intelligence

4. Knowledge Graph Layers

Layer 1 - Business Graph

Represents business architecture.

Nodes

```
Application
Domain
Capability
BusinessRule
ServiceCandidate
```

Example:

```
Card Processing
  ↓
Authorization
  ↓
Card Validation
```

Layer 2 - Application Graph

Represents application structure.

Nodes

```
Program
Job
Copybook
Transaction
API
Screen
```

Example:

```
TS2AUTH
  CALLS
TS2VALID
```

Layer 3 - Execution Graph

Represents runtime execution.

Nodes

Scheduler
Job
Step
Program
Transaction

Example:

NightlySettlementJob
EXECUTES
TS2SETL

Layer 4 - Data Graph

Represents information flow.

Nodes

Table
Column
Dataset
VSAM
MQQueue
Message

Example:

TS2AUTH
WRITES
CARD_MASTER

Layer 5 - Modernization Graph

Represents future-state architecture.

Nodes

```
Capability
Domain
ServiceCandidate
MigrationWave
RiskItem
Recommendation
```

Example:

```
Authorization Capability
  ↓
Authorization Service
```

5. Node Taxonomy

Business Nodes

Application

Examples:

- Card Processing
- Core Banking
- Billing

Attributes:

```
{
  "application_id": "",
  "application_name": "",
  "description": "",
  "owner": ""
}
```

Domain

Examples:

- Authorization
- Settlement
- Customer Management

Attributes:

```
{
  "domain_id": "",
  "domain_name": "",
  "application_id": ""
}
```

Capability

Examples:

- Card Activation
- Balance Inquiry
- Payment Processing

Attributes:

```
{
  "capability_id": "",
  "capability_name": "",
  "domain_id": ""
}
```

6. Technical Node Taxonomy

Program

Core node type.

Attributes:

```
{
  "program_id": "",
  "program_name": "",
  "language": "",
  "complexity_score": 0,
  "loc": 0
}
```

Job

Attributes:

```
{
  "job_id": "",
  "job_name": "",
  "step_count": 0
}
```

Copybook

Attributes:

```
{
  "copybook_id": "",
  "copybook_name": "",
  "field_count": 0
}
```

Table

Attributes:

```
{
  "table_id": "",
  "schema_name": "",
  "table_name": ""
}
```

Dataset

Attributes:

```
{
  "dataset_id": "",
  "dataset_name": "",
  "dataset_type": ""
}
```

7. Edge Taxonomy

Execution Relationships

CALLS
EXECUTES
STARTS
INVOKES
SCHEDULES

Examples:

TS2AUTH CALLS TS2VALID
JOB001 EXECUTES TS2AUTH

Data Relationships

READS
WRITES
PRODUCES
CONSUMES
UPDATES

Examples:

TS2AUTH WRITES CARD_MASTER
TS2AUTH READS CUSTOMER_MASTER

Structural Relationships

CONTAINS
USES
PART_OF
BELONGS_TO

Examples:

```
TS2AUTH USES CPY_CARD
```

```
Authorization BELONGS_TO CardProcessing
```

Modernization Relationships

```
BECOMES  
REPLACES  
MIGRATES_TO  
DEPRECATES
```

Example:

```
Authorization Capability  
    BECOMES  
Authorization Service
```

8. Graph Storage Model

NetworkX

Use:

```
import networkx as nx  
  
graph = nx.MultiDiGraph()
```

Reason:

Multiple relationships may exist between the same nodes.

Example:

```
TS2AUTH  
  
READS CARD_MASTER  
  
WRITES CARD_MASTER  
  
USES CPY_CARD
```

9. Node Schema

All nodes follow:

```
{
  "id": "",
  "type": "",
  "name": "",
  "attributes": {},
  "confidence": 1.0,
  "source": "",
  "evidence": []
}
```

10. Edge Schema

All edges follow:

```
{
  "relationship_type": "",
  "confidence": 1.0,
  "source_file": "",
  "line_number": 0,
  "evidence": []
}
```

11. Graph Construction Pipeline

```
Repository
  ↓
Discovery
  ↓
Parsers
  ↓
Metadata
  ↓
Validation
  ↓
Relationship Builder
  ↓
Knowledge Graph
```

↓
Reasoning Engine

12. Core Traversals

Call Chain Analysis

Question:

Who calls TS2AUTH?

Traversal:

Incoming CALLS edges

Impact Analysis

Question:

What breaks if TS2AUTH changes?

Traversal:

Forward dependency traversal

Root Program Discovery

Question:

What starts TS2AUTH?

Traversal:

Reverse traversal
until root node found

Data Lineage

Question:

Where did this dataset originate?

Traversal:

Producer → Consumer path

13. Graph Algorithms

Centrality Analysis

Calculate:

Degree Centrality

Betweenness Centrality

Closeness Centrality

Eigenvector Centrality

Purpose:

Identify critical components.

Community Detection

Algorithms:

Louvain

Girvan-Newman

Label Propagation

Purpose:

Discover:

- Domains
- Capabilities
- Service Boundaries

Connected Components

Used to identify:

Independent Subsystems

Dead Zones

Orphan Components

14. Modernization Intelligence Graph

This is the MIP differentiator.

Transformation:

```
Programs
  ↓
Capabilities
  ↓
Domains
  ↓
Services
  ↓
Migration Waves
```

Example:

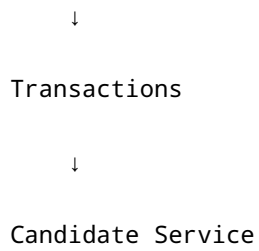
Authorization Capability

↓

Programs

↓

Tables



15. Enterprise Scale Targets

Support:

100,000+ Programs
20,000+ Jobs
500,000+ Tables
10,000,000+ Relationships

16. Graph Quality Rules

Every edge must:

- Have source evidence
- Have confidence score
- Be reproducible
- Be explainable

17. Example Queries

Programs Updating CARD_MASTER

```
MATCH (p:Program)-[:WRITES]->(t:Table)
WHERE t.name='CARD_MASTER'
RETURN p
```

Jobs Executing TS2AUTH

```
MATCH (j:Job)-[:EXECUTES]->(p:Program)
WHERE p.name='TS2AUTH'
RETURN j
```

Impact Analysis

```
MATCH path=(p:Program)-[*]->(n)
WHERE p.name='TS2AUTH'
RETURN path
```

18. Future Graph Extensions

Phase 4+

Add:

```
API
Event
MQ
IMS
Screen
Business Rule
Data Element
Service Candidate
Migration Wave
Risk Item
```

19. Success Criteria

A new engineer should be able to ask:

Show everything related to Card Activation.

And receive:

- Programs
- Jobs
- Transactions

- Tables
- Datasets
- Dependencies
- Risks
- Modernization Recommendations

without reading COBOL.

MIP Principle

Source Code stores implementation.

Metadata stores facts.

Knowledge Graph stores understanding.

Reasoning Engine generates intelligence.

Modernization Intelligence drives transformation.

End of Document